

PERSONAL STUDY NOTE

DiT

Scalable Diffusion Models with Transformers

从 U-Net 到 Transformer Diffusion Backbone

中文深度解析 / Chinese-first, English-annotated insight report



论文 Figure 1: DiT-XL/2 的 class-conditional ImageNet 样例。

论文信息 / Paper metadata	内容 / Details
作者 / Authors	William Peebles, Saining Xie
会议 / Venue	ICCV 2023; arXiv:2212.09748v2

论文信息 / Paper metadata	内容 / Details
核心任务 / Task	Class-conditional latent diffusion on ImageNet 256×256 and 512×512
核心贡献 / Contribution	以标准 Vision Transformer backbone 替换 diffusion U-Net，并系统研究模型规模、patch size 与 forward GFLOPs 的 scaling behavior。
代表模型 / Flagship	DiT-XL/2: 28 blocks, hidden 1152, 16 heads, 675M params
代表结果 / Result	ImageNet 256×256 FID 2.27 with classifier-free guidance

目录与阅读路线 / Navigation

- 1. 先读 Executive Summary，建立一句话模型：DiT = 在 VAE latent patch 上运行、由 timestep/class 通过 adaLN-Zero 调制的 Vision Transformer denoiser。
- 2. 再读第 4-6 节：沿 tensor shape 理解 $32 \times 32 \times 4$ latent 如何变为 256×1152 tokens，再如何还原为 $32 \times 32 \times 8$ 输出。
- 3. 然后读第 7-9 节：理解模型规模、patch size、GFLOPs 与 FID 的实验因果链。
- 4. 最后读批判性分析：DiT 的价值是 architecture unification 与 scalability，不是宣称 U-Net 在任何情况下都无效。

章节 / Section	核心问题 / Question
1-3	论文究竟改变了什么？为什么在 latent space 做 Transformer？
4-6	每一层如何传递数据？adaLN-Zero 为什么是关键？
7-9	如何定义 DiT-S/B/L/XL 与 /8,/4,/2？为什么 GFLOPs 比参数量更能解释质量？
10-12	训练、CFG、结果和消融分别支持哪些结论？
13-15	局限、后续影响、复现要点与自测问题。

阅读声明 / Reading disclaimer: 本文是面向理解与实现的二次技术解读，不是逐句翻译。所有论文事实、数值和原图来自本地 arXiv v2 PDF；推导性解释会明确使用“可以理解为”“工程上”等表述。

Executive Summary：先记住这八句话

- 1. DiT 的核心动作极其简洁：保留 latent diffusion 的 VAE 与 diffusion objective，只把常用的 convolutional U-Net denoiser 替换为标准 Transformer。

2. 输入不是 RGB pixel，而是冻结 VAE 产生的 spatial latent；256×256 图像对应 32×32×4 latent，因此 attention 发生在低分辨率语义空间。
3. DiT 把 latent 切成 $p \times p$ patches。DiT-XL/2 的 $p=2$ ，因此 32×32 latent 变成 $16 \times 16 = 256$ 个 token，每个 token hidden size 为 1152。
4. timestep 和 class label 不作为普通 image tokens，而是相加形成 condition vector，再由 adaLN-Zero 为每个 block 生成 shift、scale 与 residual gate。
5. adaLN-Zero 的关键不是只有 adaptive normalization，而是将 residual gate 初始化为零，使每个深层 block 在训练开始时近似 identity mapping。
6. 论文建立的主要经验结论是：增加 Transformer forward GFLOPs，无论通过加深/加宽 backbone 还是减小 patch size 增加 tokens，FID 都会稳定下降。
7. DiT-XL/2 在 ImageNet 256×256 上达到 FID 2.27，但该结果依赖 classifier-free guidance、长训练和特定 VAE decoder；不能只归因于架构。
8. DiT 的历史价值是证明 diffusion backbone 可以标准化为 Transformer，从而直接继承语言模型和 ViT 领域的 scaling、并行化与架构创新。

一句话抽象 / One-line abstraction: DiT is a ViT-style latent denoiser whose image tokens are globally mixed by self-attention and whose diffusion/class conditions modulate every residual block through zero-initialized adaptive LayerNorm.

1. 论文要解决什么问题 / Problem Definition

1.1 为什么 diffusion 长期依赖 U-Net

早期 DDPM 继承了图像生成领域成熟的 U-Net：多尺度 downsample/upsample、skip connections、ResNet blocks 与局部卷积天然适合处理空间信号。后来 attention 被插入低分辨率层，但网络骨架仍然是 U-Net。问题是，当 NLP 与视觉识别都因为 Transformer 获得更清晰的 scaling law 时，diffusion generation 是否真的必须保留 U-Net 的强 inductive bias？

DiT 将问题改写为一个 architecture scaling question：如果固定 latent diffusion、训练目标和数据，只替换 denoiser backbone，标准 Transformer 是否能稳定扩大，并把更多 forward compute 转化为更好的生成质量？

1.2 论文真正验证的假设

假设 / Hypothesis	论文如何验证
U-Net 不是高质量 diffusion 的必要条件	在相同 ImageNet latent-diffusion setting 下训练纯 Transformer denoiser。
Transformer 可通过规模扩展持续改善生成质量	扫描 S/B/L/XL 四种规模、/8,/4,/2 三种 patch size，共 12 个主模型。
forward GFLOPs 是比参数量更有解释力的复杂度指标	保持参数近似不变，仅减小 patch size 提高 token 数与计算量，观察 FID 改善。
条件注入方式会显著影响训练	对比 in-context、cross-attention、adaLN 与 adaLN-Zero。

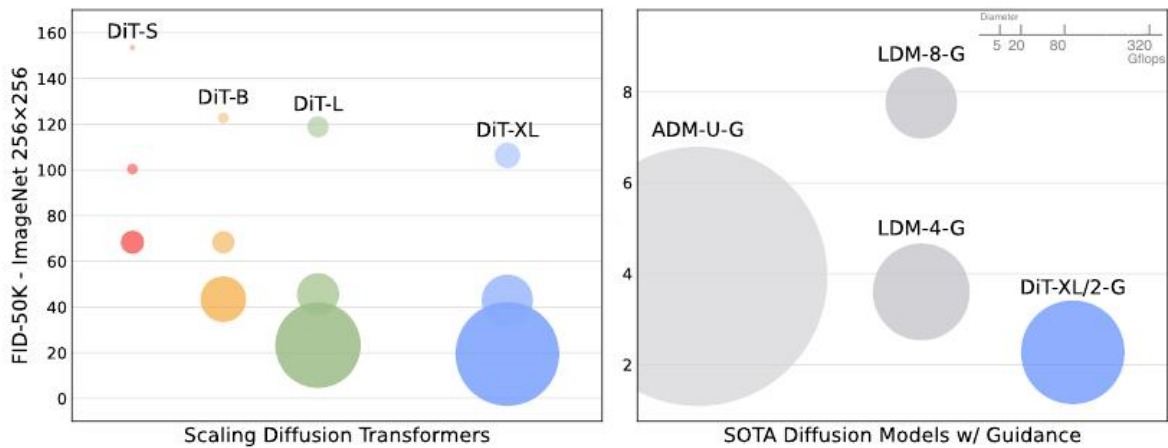


Figure 2. **ImageNet generation with Diffusion Transformers (DiTs).** Bubble area indicates the flops of the diffusion model. *Left:* FID-50K (lower is better) of our DiT models at 400K training iterations. Performance steadily improves in FID as model flops increase. *Right:* Our best model, DiT-XL/2, is compute-efficient and outperforms all prior U-Net-based diffusion models, like ADM and LDM.

论文 Figure 2：左图展示 DiT scaling，右图将 DiT-XL/2 与 U-Net-based ADM/LDM 对比。

2. 预备知识 / Background

2.1 DDPM：网络到底在预测什么

训练时从真实 latent z_0 出发，随机选择 timestep t 并加入高斯噪声，得到 z_t 。网络接收 z_t 、 t 和条件 y ，预测加入的噪声 ϵ ，同时学习反向过程方差。

$$z_t = \sqrt{\alpha_t} z_0 + \sqrt{1-\alpha_t} \epsilon, \quad \epsilon \sim N(0, I)$$

$$L_{\text{simple}} = || \epsilon_{\theta}(z_t, t, y) - \epsilon ||_2^2$$

为什么预测 noise 而不是直接预测 image？因为每个 timestep 的监督目标被标准化为已知高斯噪声，训练稳定；通过已知的前向过程公式，可以由噪声预测恢复 reverse mean。DiT 没有改变 diffusion mathematics，它只改变函数 ϵ_{θ} 的网络实现。

2.2 Latent Diffusion : 为什么不直接处理 256×256 pixels

如果对 256×256 pixel token 做全局 attention，序列长度和计算成本都很高。论文使用现成的冻结 VAE，将图像压缩 8 倍：

$$\mathbf{x} \in \mathbb{R}^{(256 \times 256 \times 3)} \rightarrow \mathbf{z} = \mathbf{E}(\mathbf{x}) \in \mathbb{R}^{(32 \times 32 \times 4)}$$

这使 DiT 成为 hybrid architecture：图像压缩与重建仍由 convolutional VAE 完成，真正的 diffusion denoising backbone 才是 Transformer。因此“pure Transformer diffusion”准确含义是 denoiser 纯 Transformer，不是整个生成系统没有卷积。

2.3 Classifier-Free Guidance (CFG)

$$\epsilon_{\text{cfg}} = \epsilon_{\text{uncond}} + s (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$$

训练中以 10% 概率丢弃 class label，用 learned null embedding 训练 unconditional branch。采样时同一 z_t 分别做 conditional 和 unconditional forward，再沿条件方向放大。guidance scale 越大通常语义更强、视觉保真更高，但 diversity 会下降。附录说明论文只对 latent 的前三个通道执行 guidance，这是一个容易忽略的复现细节。

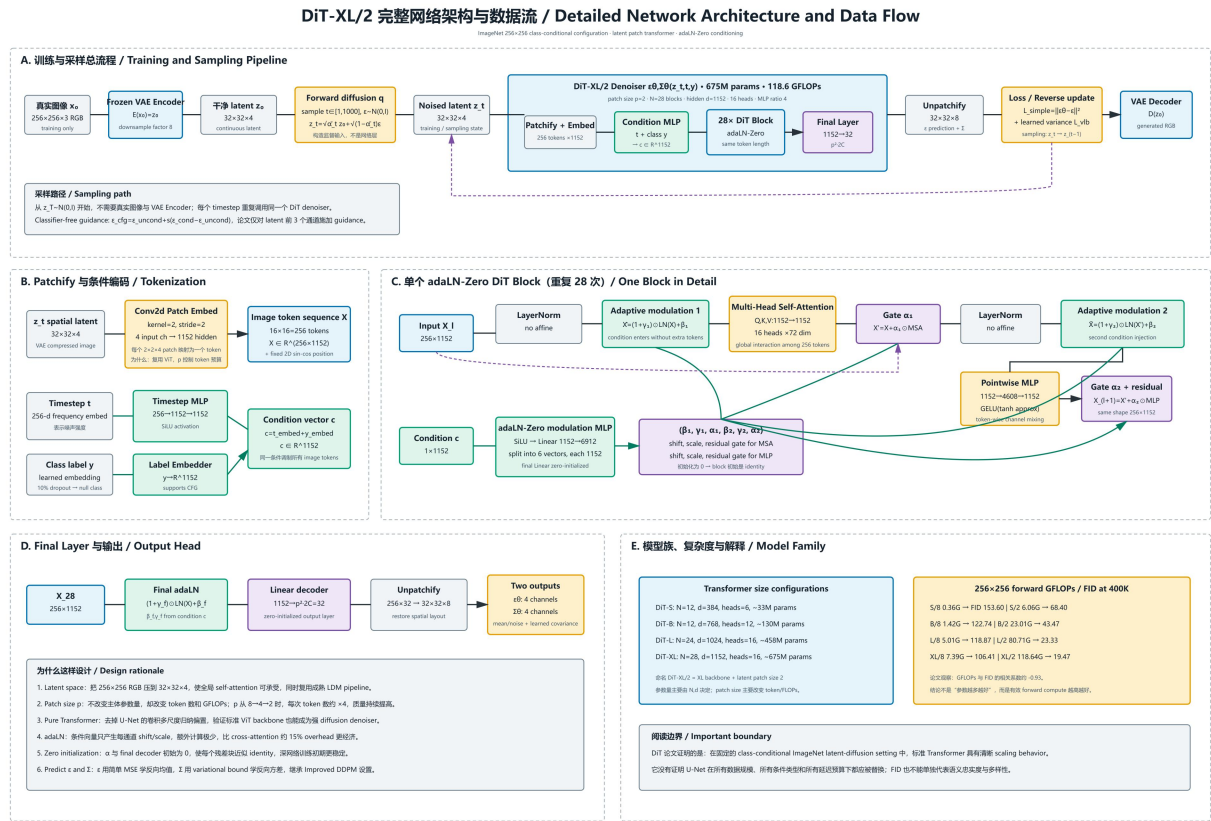
3. 从 U-Net 到 DiT : 架构思想变化

维度	Diffusion U-Net	DiT
基本单元	ResNet + Conv + multi-scale attention	ViT Transformer block
空间组织	显式 down/up sampling hierarchy	固定 token resolution，所有 block 等宽
长程依赖	主要在低分辨率 attention 层	每一层都可全局 self-attention
条件注入	time/class embedding 注入 ResNet normalization	adaLN-Zero 生成 shift/scale/gate
复杂度扩展	通道数、层数、resolution stage	depth N、width d、token count T

维度	Diffusion U-Net	DiT
工程优势	成熟、局部先验强、不同尺度自然	结构统一、并行友好、易继承 Transformer 生态

DiT 的创新不在 attention 公式，而在于把 diffusion denoiser 重构成几乎标准的 ViT design space，并对 conditioning、output head 和 initialization 做最小但关键的修改。

4. DiT 总体网络架构 / Overall Architecture



自绘 Figure A: DiT-XL/2 完整中英双语网络架构与 tensor 数据流。SVG 同目录提供，可无限放大。

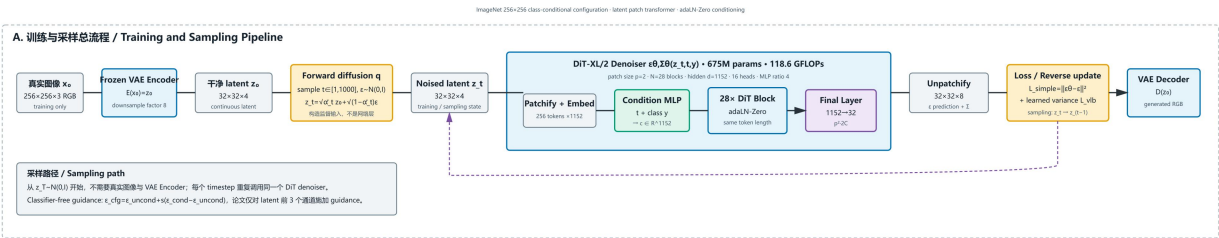


Figure A1 局部放大：训练、采样、VAE、DiT denoiser 与 reverse update 的完整端到端数据流。

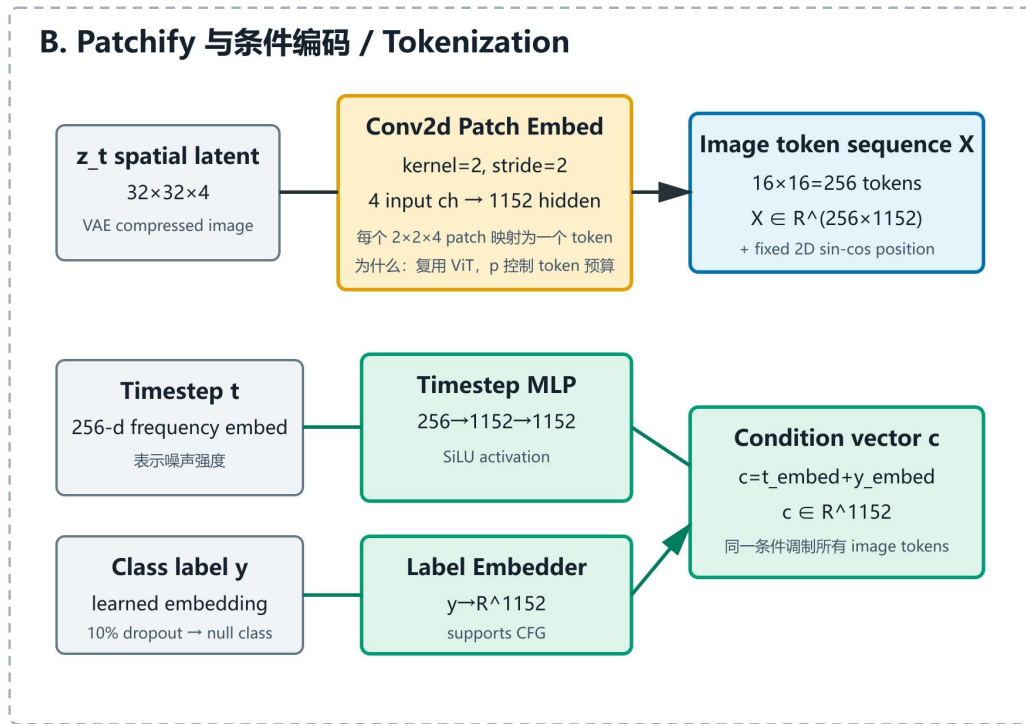


Figure A2 局部放大: latent patchify、timestep embedding、class embedding 与 condition vector。

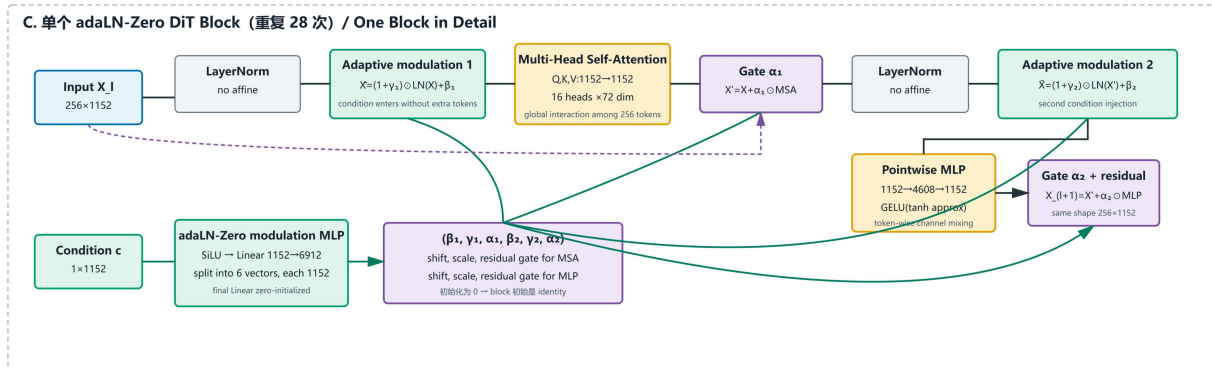


Figure A3 局部放大: 单个 adaLN-Zero block 的六路条件参数、attention、MLP 和 residual gates。

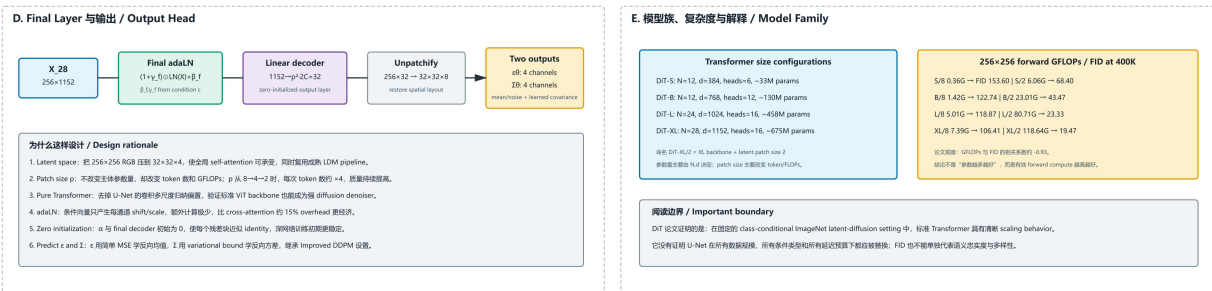


Figure A4 局部放大: final output head、模型族、GFLOPs/FID 与设计理由。

这张图需要分四条路径阅读:

- Data path: $z_t \rightarrow \text{patchify} \rightarrow 28 \times \text{DiT blocks} \rightarrow \text{final layer} \rightarrow \text{unpatchify} \rightarrow \text{epsilon/Sigma prediction}$ 。

- Condition path: timestep embedding + class embedding $\rightarrow c \rightarrow$ 每个 block 的 6d adaLN-Zero parameters.
- Residual path: MSA 和 MLP 的输出分别乘 α_1/α_2 , 再加回 token stream.
- Diffusion loop: 网络输出被 sampler 用于 $z_t \rightarrow z_{(t-1)}$, 同一网络在所有 timesteps 反复调用。

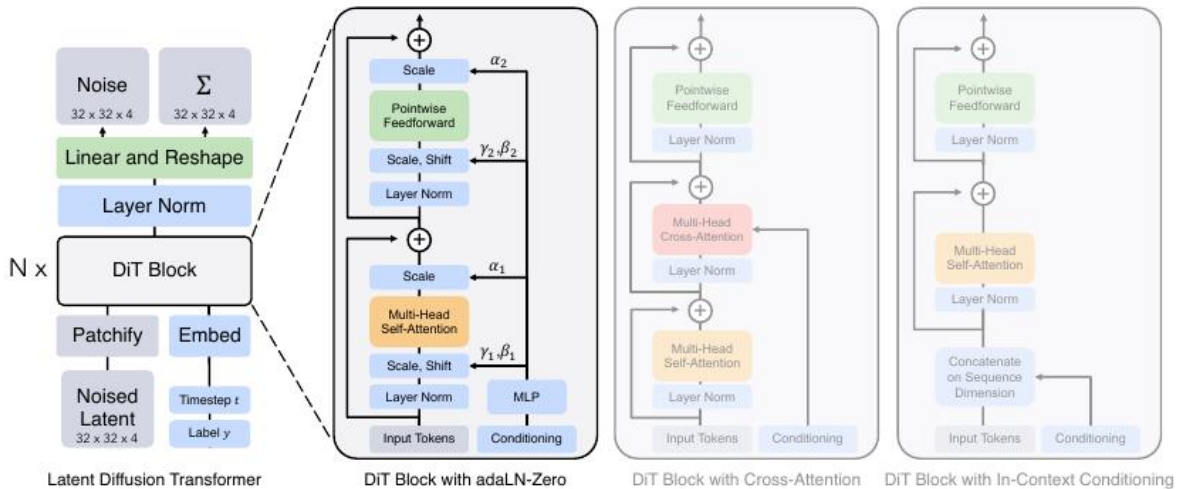


Figure 3. **The Diffusion Transformer (DiT) architecture.** *Left:* We train conditional latent DiT models. The input latent is decomposed into patches and processed by several DiT blocks. *Right:* Details of our DiT blocks. We experiment with variants of standard transformer blocks that incorporate conditioning via adaptive layer norm, cross-attention and extra input tokens. Adaptive layer norm works best.

论文 Figure 3 原图: 左侧为总体 DiT, 右侧比较三种条件 block; 本文架构图在此基础上展开到具体 tensor 与层。

5. 网络架构逐层详解：每层是什么、为什么这样设计

5.1 输入 VAE latent : 32×32×4

对 256×256 图像, 冻结 VAE encoder 输出 z_t shape 32×32×4。4 个 channel 不是 RGB, 而是 learned latent features。选择 latent space 的直接收益是把空间边长缩小 8 倍、位置数缩小 64 倍; 代价是最终质量受 VAE reconstruction ceiling 限制。论文后面对 decoder 的消融正说明这个边界。

5.2 Patchify layer : Conv2d(kernel=p, stride=p)

Patchify 可等价实现为不重叠卷积。对于 DiT-XL/2, $p=2$: 每个 patch 包含 $2 \times 2 \times 4 = 16$ 个值, 经 linear projection 映射到 1152 维。32×32 latent 被切成 $16 \times 16 = 256$ tokens。

$$T = (I/p)^2; \text{ DiT-XL/2: } T = (32/2)^2 = 256$$

为什么 patch size 是最关键的计算旋钮? Self-attention 复杂度含 T^2d , MLP 复杂度含 Td^2 。p 减半会让 T 增加 4 倍, 因此 attention 项约增加 16 倍, 同时参数量几乎不变。这让论文可以区分 parameter scaling 和 compute scaling。

5.3 Fixed 2D sine-cosine positional embedding

Transformer 不天然知道 token 的二维位置，因此将 fixed 2D sin-cos position embedding 加到 patch tokens。作者遵循 ViT 习惯而非学习位置表。固定位置编码不增加训练参数，并为不同模型提供一致的位置基准；但原始实现仍面向固定 resolution，不代表可以无代价外推到任意尺寸。

5.4 Timestep embedding 与 class embedding

timestep t 先使用 256 维 frequency embedding，再经过两层 MLP 投影到 hidden size d ；class label 使用 learned embedding 得到同维向量。两者直接相加形成 condition c 。 t 表示当前噪声水平， y 表示应该生成的类别；网络需要同时知道“现在有多脏”和“最终要生成什么”。

5.5 adaLN-Zero modulation MLP : 1152→6×1152

每个 DiT block 都有自己的 conditioning MLP：SiLU(c) 后通过 Linear 输出 6 个 d 维向量：beta1、gamma1、alpha1、beta2、gamma2、alpha2。前两组用于 attention 前的 normalization 与 residual gate，后三组用于 MLP 前的 normalization 与 gate。

$$\text{Modulate}(x; \text{gamma}, \text{beta}) = x * (1 + \text{gamma}) + \text{beta}$$

为什么不是普通 LayerNorm？普通 LN 对每个 token 使用固定 affine parameters；adaLN 让 affine parameters 由当前 timestep/class 动态产生，因此同一组 Transformer weights 可以在不同噪声阶段执行不同 denoising function。

为什么要 alpha gate？alpha 直接控制 attention/MLP residual 写回主干的幅度。将 modulation Linear 初始化为零后，beta=gamma=alpha=0，block 初始行为变成 $X_{\text{out}} \approx X_{\text{in}}$ 。深达 28 层的网络因此从稳定 identity mapping 开始学习，而不是一开始让随机 residual 连续扰动 token。

5.6 Multi-Head Self-Attention

DiT-XL 使用 16 个 heads，hidden 1152，因此每个 head dimension 为 72。Q/K/V 都由 1152 维 token 投影得到。所有 256 个 latent patch tokens 在每个 block 中彼此可见，这使网络能够在固定 latent resolution 上建立全局结构。

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_h}) V$$

为什么这里不需要 text cross-attention？原论文是 class-conditional ImageNet，condition 是两个全局向量 t 与 y ，而不是长度可变的文本序列。adaLN 是更低成本且足够的条件方式。后来的 text-to-image DiT/MDT/MM-DiT 才更需要文本 token 交互。

5.7 Pointwise MLP : $d \rightarrow 4d \rightarrow d$

每个 token 独立经过两层 feed-forward network: $1152 \rightarrow 4608 \rightarrow 1152$, 使用 GELU(tanh approximation)。

Attention 负责 token-to-token spatial mixing; MLP 负责每个 token 内部的 channel mixing。两者共同构成标准 Transformer block。

5.8 Final adaLN + Linear decoder

28 个 blocks 后, 再使用 condition c 生成 final shift/scale, 对 tokens 做最后一次 adaptive normalization。Linear 将每个 token 从 1152 维映射到 $p \times p \times 2C$ 。 $C=4$, $p=2$, 所以输出 32 维。Unpatchify 后得到 $32 \times 32 \times 8$, 其中 4 channels 是噪声预测, 4 channels 是 learned covariance prediction。

为什么输出层也 zero-init? 训练开始时网络预测接近零, 避免随机大幅输出破坏 diffusion objective; 随后逐渐学习有效噪声场。

6. 四种条件注入方式 : 为什么 adaLN-Zero 胜出

方式	数据流	计算代价	核心问题
In-context	把 t, y 作为两个额外 tokens 与 image tokens 拼接	低到中	条件与图像 token 共享 self-attention, 但条件可能被稀释。
Cross-attention	image self-attention 后增加条件 cross-attention	最高, 约+15% GFLOPs	表达力强, 但 class/timestep 仅两个向量时显得昂贵。
adaLN	c 生成每通道 shift/scale	最低	高效, 但随机初始化 residual 深堆叠仍可能不稳定。
adaLN-Zero	adaLN + 两个 residual gates, 零初始化	几乎与 adaLN 相同	兼顾条件控制、低开销和 identity initialization。

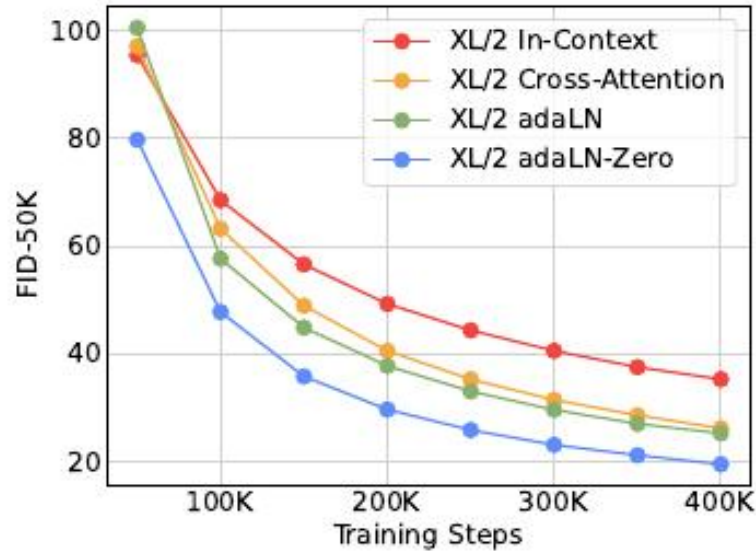


Figure 5. Comparing different conditioning strategies. adaLN Zero outperforms cross-attention and in-context conditioning at a stages of training.

论文 Figure 5: adaLN-Zero 在整个 400K 训练过程中均优于其他条件机制。

这组消融的真正结论不是“cross-attention 不好”，而是：当条件只有 timestep 与 class embedding 时，用条件去调制 normalization 比维护独立 cross-attention 序列更匹配问题结构。文本生成的条件更丰富，结论不能直接照搬。

7. DiT 模型族：S/B/L/XL 与 /8,/4,/2

Model	Layers N	Hidden size d	Heads	Gflops ($T=32, p=4$)
DiT-S	12	384	6	1.4
DiT-B	12	768	12	5.6
DiT-L	24	1024	16	19.7
DiT-XL	28	1152	16	29.1

Table 1. Details of DiT models. We follow ViT [10] model configurations for the Small (S), Base (B) and Large (L) variants; we

论文 Table 1: S/B/L/XL 的 depth、hidden size、heads 与基准 GFLOPs。

配置	Layers N	Hidden d	Heads	Params approx.
DiT-S	12	384	6	33M

配置	Layers N	Hidden d	Heads	Params approx.
DiT-B	12	768	12	130M
DiT-L	24	1024	16	458M
DiT-XL	28	1152	16	675M

斜杠后的数字是 latent patch size，不是 image patch size。例如 DiT-XL/2 表示 XL backbone 与 2×2 latent patches。对 256×256 RGB，latent 是 32×32，因此 /8、/4、/2 分别产生 16、64、256 tokens。

模型名	Token grid	Tokens T	400K GFLOPs	400K FID (no CFG)
DiT-XL/8	4×4	16	7.39	106.41
DiT-XL/4	8×8	64	29.05	43.01
DiT-XL/2	16×16	256	118.64	19.47

关键洞察 / Key insight: Patch size scaling 几乎不增加 backbone 参数，却显著增加 token computation；因此论文观察到同参数模型也能通过更多 GFLOPs 获得更低 FID。

8. 训练配置与复现要点 / Training

项目	论文设置	为什么重要
Dataset	ImageNet class-conditional, 256 ² /512 ²	受控 benchmark，便于与 ADM/LDM 对比。
Optimizer	AdamW, lr=1e-4, no weight decay	所有规模共享设置，减少调参干扰。
Batch size	256	统一 global batch。
Augmentation	horizontal flip only	生成任务避免强判别式增强。

项目	论文设置	为什么重要
Diffusion	1000-step linear variance $1e-4 \rightarrow 2e-2$	继承 ADM 设置。
Sampling for FID	250 DDPM steps	比较时需保持 sampler 一致。
EMA	用于最终采样	生成模型常见稳定权重。
Training length	XL/2 256^2 trained 7M steps; 512^2 3M	SOTA 结果远超 400K scaling sweep。

论文的重要实验纪律是：12 个 scaling 模型在 400K 时统一比较，而最终 DiT-XL/2 继续训练数百万步。阅读结果时必须区分 architecture sweep 与 final SOTA model，否则会误以为 FID 2.27 是 400K 训练得到。

9. Scaling 规律：GFLOPs 为什么成为论文主角

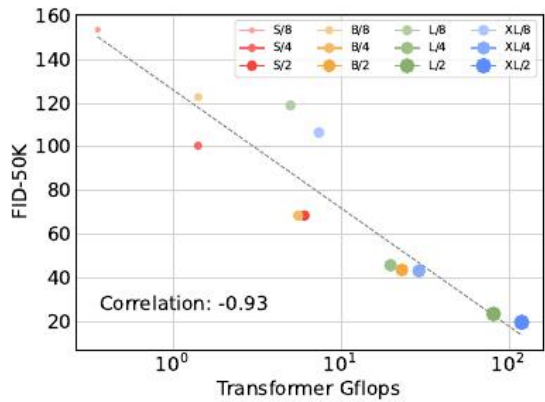


Figure 8. **Transformer GFlops are strongly correlated with FID.** We plot the GFlops of each of our DiT models and each model’s FID-50K after 400K training steps.

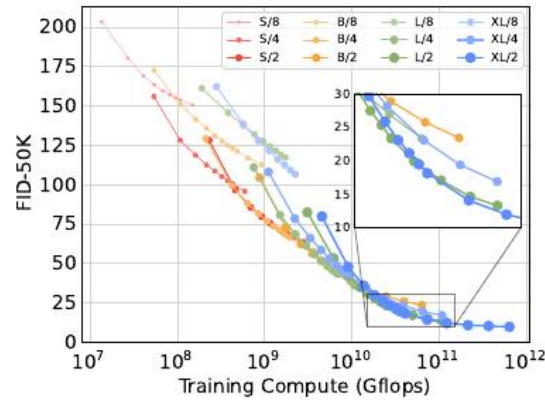


Figure 9. **Larger DiT models use large compute more efficiently.** We plot FID as a function of total training compute.

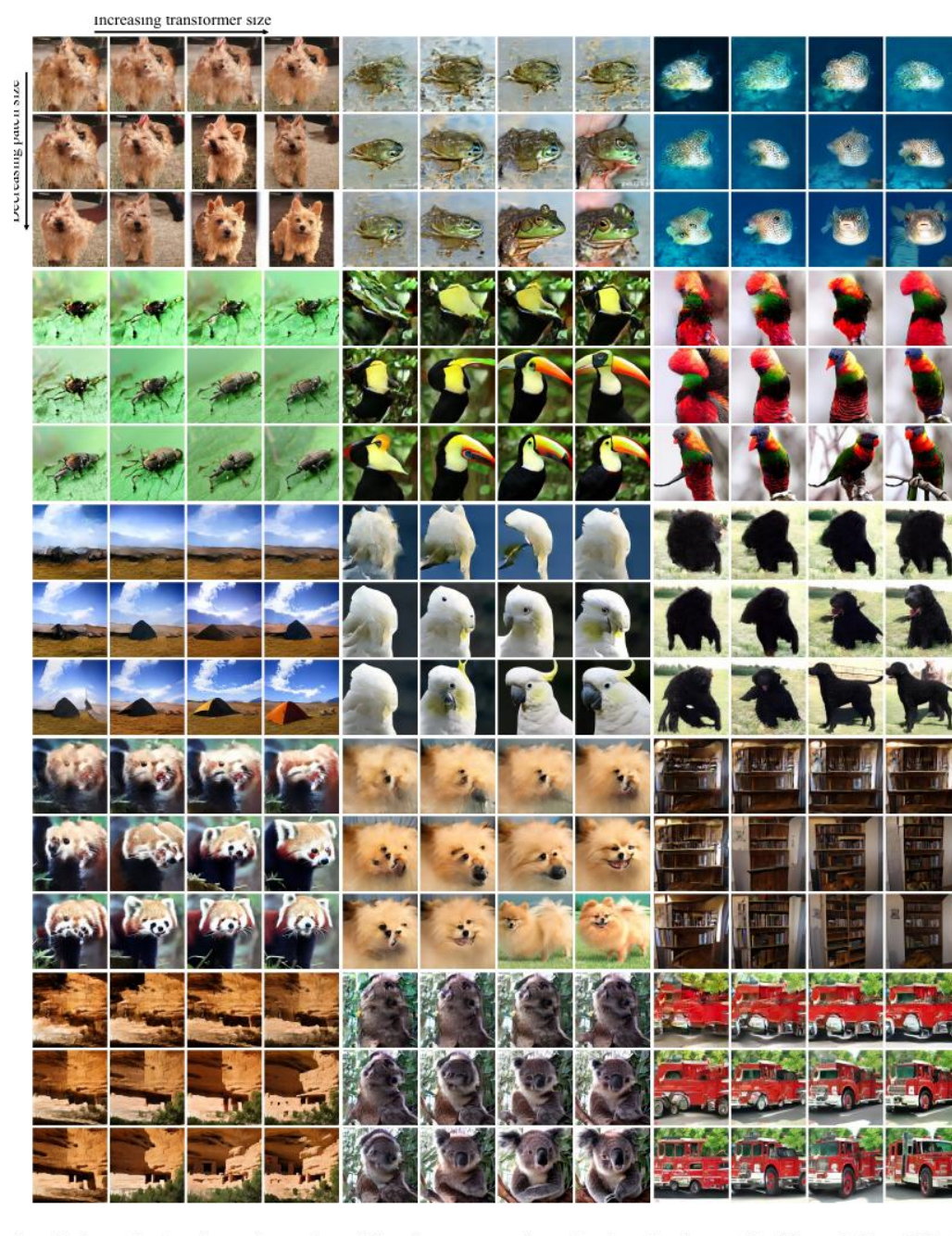
论文 Figures 8-9: GFLOPs 与 FID 强负相关; 大模型在总训练 compute 下最终更高效。

参数量不是图像生成网络成本的完整代理，因为输入 resolution/token count 会极大改变实际计算。DiT 的实验把 forward GFLOPs 作为横轴，得到约 -0.93 的相关系数：模型每次 forward 做得更多，FID 通常越低。

更细的解释有三层：

- 增加 depth/width：提高参数容量与每个 token 的表示能力。
- 减小 patch size：提高空间 token 数，让网络在 latent 上保留更细粒度的局部结构。
- 同样总训练 compute 下，大模型虽然每步更贵，但更快达到较低 FID，最终可能比小模型长时间训练更 compute-efficient。

但 GFLOPs 只是 theoretical forward complexity，不等于真实 latency。Attention kernel、memory bandwidth、hardware utilization、batch size 与 precision 都会改变 wall-clock efficiency。论文自己也承认“golden complexity metric”仍有争议。



论文 Figure 7: 固定初始噪声与类别，横向增加模型规模、纵向减小 patch size，结构与纹理逐渐成形。

10. 主要结果与效果图 / Results

10.1 ImageNet 256×256

DiT-XL/2 在 7M steps 后, 无 guidance FID 为 9.62; 使用 classifier-free guidance 后达到 FID 2.27、sFID 4.60、IS 278.24、Precision 0.83、Recall 0.57。它超过论文所列先前 diffusion baselines。

FID 改善伴随典型 CFG trade-off: 更大的 guidance 提高视觉语义一致性和 precision, 但降低 diversity/recall。结果不能只看最小 FID, 还应结合 precision-recall 与未筛选样例。

10.2 ImageNet 512×512

512² 图像经过 VAE 后 latent 为 64×64×4; DiT-XL/2 因此处理 32×32=1024 tokens, forward GFLOPs 增至 524.6。该模型训练 3M steps, 论文报告 FID 3.04 with guidance。分辨率翻倍使 token 数 4 倍, attention 成本迅速增长, 预示高分辨率 DiT 后续必须研究更高效 attention 或更强 latent compression。



论文 Figure 11: 512² 与 256² DiT-XL/2 selected samples。

10.3 Curated 与 uncurated 样例要分开



Figure 14. Uncurated 512×512 DiT-XL/2 samples.



Figure 15. Uncurated 512×512 DiT-XL/2 samples.

论文 Figures 14-15: 512^2 uncurated samples, class labels 为 arctic wolf 与 volcano。

首页 Figure 1 是 selected samples, 附录 Figures 14-33 才是不同类别与 guidance 下的 uncurated samples。后者能看到类内重复、构图偏好与失败模式, 更适合判断真实分布。较高 CFG 会让类别特征更集中, 但也可能让样本之间更相似。

11. VAE Decoder : 为什么它会影响 FID

DiT 生成的是 latent, 最终像素必须由 VAE decoder 还原。因此最终 FID 是 Transformer denoiser 与 decoder 共同作用的结果。论文比较原始 MSE decoder、EMA decoder 与 fine-tuned decoder, 并使用 ft-EMA 获得最佳 FID 2.27。

这带来一个重要方法论：当评价 latent generative model 时，不应把所有 pixel-space quality 都归因于 latent backbone。VAE reconstruction artifacts、颜色与纹理偏差都会影响 FID。模型对比必须说明 encoder/decoder checkpoint。

12. Critical Reading：论文证明了什么，没有证明什么

判断	证据强度	解释
Transformer 可以替代 U-Net 成为强 diffusion backbone	强	12 模型系统扫描、条件消融与 SOTA 结果共同支持。
增加 DiT forward compute 可持续改善 ImageNet FID	强	模型规模与 patch size 两条独立扩展路径均一致，相关系数高。
adaLN-Zero 是 class-conditional DiT 的最佳条件方式	中到强	固定 XL/2 对比清晰，但只测试特定数据与全局条件。
Transformer 在所有 diffusion 任务中都优于 U-Net	未证明	没有覆盖小数据、低延迟、restoration、video 或 dense conditioning。
GFLOPs 可完全代表实际效率	未证明	理论计算不包含 memory、kernel 和 hardware 利用率。
FID 2.27 表示生成分布全面优越	不充分	FID 受 decoder、CFG 与统计实现影响，不能单独衡量多样性和语义错误。

一个容易忽略的 limitation 是数据规模与任务形式：ImageNet class labels 只有 1000 个离散类别，condition 结构远简单于 text-to-image。DiT 的 backbone scaling 结论非常重要，但原论文并未解决长文本对齐、复杂布局、文字渲染、视频时序或超高分辨率 memory。

13. DiT 对后续生成模型的影响

DiT 提供了一个标准底座，使后续研究可以把注意力集中到 tokenization、conditioning、attention efficiency、flow matching 和 multimodal fusion，而不是继续围绕 U-Net stage 手工设计。

后续方向	从 DiT 继承的核心	进一步改变
PixArt / text-to-image DiT	latent patch Transformer scaling	加入文本 cross-attention、更高效训练与数据策略。
Stable Diffusion 3 / MM-DiT	Transformer diffusion backbone	图像与文本双流 joint attention。
FLUX	大规模 multimodal DiT / rectified flow	double-stream + single-stream blocks、更大数据与文本编码器。
Video DiT	spatial latent tokens	增加 temporal tokens/attention 与时空分解。
DreamSR 等 restoration	冻结预训练 DiT prior	增加 ControlNet/LoRA 和输入图像条件。

从你刚学完的 DreamSR 反看 DiT：DreamSR 中 Frozen FLUX DiT 的祖先思想就在这里。DiT 证明 Transformer 可以做 denoiser；MM-DiT/FLUX 把 class condition 扩展为文本 token；DreamSR 再在 FLUX 旁边增加 patch-aware MM-ControlNet 和 restoration LoRA。

14. 实现蓝图 / Implementation Blueprint

14.1 最小 DiT-XL/2 forward

- 接收 $z_t [B,4,32,32]$ 、 $t [B]$ 、 $y [B]$ 。
- PatchEmbed Conv2d(4,1152,kernel=2,stride=2)，flatten 为 $[B,256,1152]$ ，加入固定 2D position embedding。
- TimestepEmbedder: $\text{frequency_embed}(t,256) \rightarrow \text{Linear}(256,1152) \rightarrow \text{SiLU} \rightarrow \text{Linear}(1152,1152)$ 。
- LabelEmbedder: $\text{Embedding}(1001,1152)$ ，额外一个 null class 支持 10% label dropout。
- $c=t_embed+y_embed$ 。
- 循环 28 个 DiTBlock：modulation(c)→6 chunks；执行 modulated LN→MSA→gated residual→modulated LN→MLP→gated residual。
- FinalLayer: final modulation(c)→LN→Linear(1152,32)。
- reshape/unpatchify 为 $[B,8,32,32]$ ，拆为 epsilon 与 variance prediction。

14.2 伪代码 / Pseudocode

```
x = patch_embed(z_t) + pos_embed          # [B, 256, 1152]
c = time_embed(t) + label_embed(y)        # [B, 1152]
for block in blocks:                       # 28 blocks
    b1, g1, a1, b2, g2, a2 = block.mod(c).chunk(6, dim=1)
    x = x + a1[:,None,:] * block.attn(modulate(block.norm1(x), b1, g1))
    x = x + a2[:,None,:] * block.mlp(modulate(block.norm2(x), b2, g2))
x = final_layer(x, c)                      # [B, 256, 32]
out = unpatchify(x)                        # [B, 8, 32, 32]
```

14.3 容易实现错的地方

- modulate 的标准形式是 $x \cdot (1 + \text{scale}) + \text{shift}$ ，不是 $x \cdot \text{scale} + \text{shift}$ 。
- adaLN-Zero modulation Linear 和 final output Linear 都要 zero initialization。
- LayerNorm 使用 `elementwise_affine=False`，因为 affine 由 condition 动态提供。
- 输出 channel 是 2C，不是 C；论文同时预测 noise 与 covariance。
- FID 复现必须匹配 VAE decoder、sampling steps、CFG channel subset、sample count 与 evaluator。
- /2 指 latent patch size；不要误写成 RGB image 上 2x2 patch。

15. Reproducibility Audit / 复现性审计

项目	可复现状态
论文与附录	本地已保存 arXiv v2 PDF，共 25 页，包含详细模型表、loss curves、VAE decoder 消融和未筛选样例。
官方代码	论文提供 project/code link；DiT 后续已有广泛公开实现。具体运行仍需锁定 commit 与依赖。
架构参数	S/B/L/XL、patch sizes、block type、timestep embedding、MLP ratio、heads 和 zero-init 均披露充分。
训练成本	最大模型使用 TPU v3-256，数百万 steps；个人完整复现实验成本很高。
评价敏感项	VAE decoder、CFG、采样步数和 FID implementation 会显著改变结果。

项目	可复现状态
建议复现层级	先做 CIFAR/ImageNet 子集上的 DiT-S/2 correctness, 再验证 adaLN-Zero identity init, 最后再扩规模。

16. Glossary / 中英文术语表

术语	English	解释
DiT	Diffusion Transformer	以 Transformer 作为 diffusion denoiser backbone。
Latent patch	潜空间图块	从 VAE latent 而不是 RGB image 切出的 patch。
Patchify	图块化	将二维 latent 映射为 token sequence。
adaLN	Adaptive LayerNorm	由 condition 动态产生 LN 的 shift/scale。
adaLN-Zero	零初始化自适应层归一化	在 adaLN 基础上增加 residual gate 并初始化为 0。
GFLOPs	十亿浮点操作	论文用于衡量单次 forward 理论复杂度。
CFG	Classifier-Free Guidance	组合 conditional/unconditional 预测增强条件一致性。
FID	Fréchet Inception Distance	比较真实与生成特征分布, 越低越好。
Unpatchify	逆图块化	把 token 输出重新排列为空间 latent。
Learned covariance	学习反向方差	除 noise/mean 外同时预测 reverse process variance。

17. Formula and Shape Sheet / 公式与张量速查

- RGB x: [B,3,256,256] → VAE latent z: [B,4,32,32]
- DiT-XL/2 patch embed: [B,4,32,32] → [B,1152,16,16] → [B,256,1152]
- Condition c: timestep [B,1152] + label [B,1152]
- Block modulation: [B,1152] → [B,6912] → $6 \times [B,1152]$
- MSA: 16 heads, head dimension 72
- MLP: 1152 → 4608 → 1152
- Final token output: [B,256,32] because $p^2 \cdot 2C = 4 \cdot 8 = 32$
- Unpatchify: [B,256,32] → [B,8,32,32] = $\epsilon[4ch] + \sigma[4ch]$

18. Self-test Questions / 自测问题

13. 为什么 DiT 被称为 latent diffusion Transformer，而不是 pixel-space Transformer？
14. 对 256×256 图像，DiT-XL/2 为什么是 256 个 tokens？请写出计算过程。
15. 减小 patch size 为何几乎不增加参数量，却大幅提高 GFLOPs？
16. timestep embedding 和 class embedding 在哪里融合？它们是否作为 image tokens 进入 attention？
17. adaLN-Zero 输出的 6 个向量分别控制什么？
18. 为什么 zero-initialized residual gate 能让深网络更容易训练？
19. DiT-XL/2 final Linear 为什么输出 32 维，而不是 16 维或 4 维？
20. 论文为什么更强调 GFLOPs 而不是 parameters？这个指标有什么局限？
21. FID 2.27 受到哪些非 backbone 因素影响？
22. 为什么原论文中 adaLN 优于 cross-attention，但文本到图像模型仍广泛使用 cross-attention/MM-DiT？
23. DreamSR 中的 FLUX DiT 与原始 DiT 在 condition 和 block 结构上有哪些演化？
24. 若把 DiT 用于 4K 图像，token 数和 attention 成本会出现什么问题？

19. Final Takeaways / 最终总结

DiT 的贡献看似只是“把 U-Net 换成 Transformer”，但其深层价值是把 diffusion backbone 纳入统一的 Transformer scaling framework。作者没有依赖复杂多尺度结构，而是用 VAE latent、patchify、标准 Transformer、adaLN-Zero 与线性输出头建立了一条足够简单、可系统扩展的基线。

最值得记住的三个设计原则是：第一，区分 model capacity 与 token compute，patch size 是独立于参数数量的重要质量旋钮；第二，conditioning 方式必须匹配条件结构，简单全局条件适合低成本 adaptive normalization；第三，深残差生成模型的 initialization 是架构的一部分，adaLN-Zero 的成功不能只归因于 LayerNorm 调制。

从技术史看，DiT 是后来 PixArt、Stable Diffusion 3、FLUX、视频生成模型和大量 restoration DiT 的共同起点。理解这篇论文后，再看 MM-DiT、ControlNet 或 DreamSR 时，最基础的数据流已经清楚：图像被转成 latent tokens，Transformer 学习 timestep-dependent vector field/noise field，外部条件决定每层如何更新这些 tokens。

References / 参考资料

- [1] William Peebles, Saining Xie. Scalable Diffusion Models with Transformers. ICCV 2023. arXiv:2212.09748v2.
- [2] Jonathan Ho et al. Denoising Diffusion Probabilistic Models. NeurIPS 2020.
- [3] Prafulla Dhariwal, Alex Nichol. Diffusion Models Beat GANs on Image Synthesis. NeurIPS 2021.
- [4] Robin Rombach et al. High-Resolution Image Synthesis with Latent Diffusion Models. CVPR 2022.
- [5] Alexey Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR 2021.
- [6] Jonathan Ho, Tim Salimans. Classifier-Free Diffusion Guidance. 2022.